

Claude Certified Architect — Foundations

Complete Exam Cheat Sheet & Study Reference

Exam Overview

Parameter	Value
Question type	Multiple choice (1 correct out of 4)
Scale	100–1000 points
Passing score	720
Guessing penalty	None — always answer every question!
Scenarios in exam	4 out of 8 randomly selected

Domain Weights — Know Where to Focus

Domain	Weight
1. Agent Architecture and Orchestration	27% — Highest weight!
2. Tool Design and MCP Integration	18%
3. Claude Code Configuration and Workflows	20%
4. Prompt Engineering and Structured Output	20%
5. Context Management and Reliability	15%

Domain 1: Agent Architecture & Orchestration (27%)

1.1 The Agentic Loop

The loop is driven by `stop_reason` — never by parsing text or counting iterations:

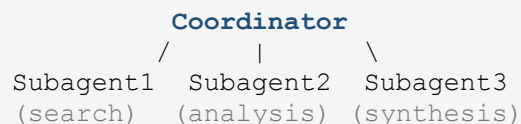
```
while True:
    response = claude_api(messages, tools)
    if response.stop_reason == 'end_turn': # ← STOP here
        return response.content
    elif response.stop_reason == 'tool_use': # ← execute tool, append result, loop
        result = execute_tool(response.tool_call)
        messages.append(tool_result(result))
```

stop_reason	Meaning	Action
end_turn	Task complete	Return answer to user
tool_use	Wants to call a tool	Execute tool, add result, loop back
max_tokens	Response truncated	Increase max_tokens
stop_sequence	Stop seq hit	Handle per app logic

⚠ Anti-patterns (wrong answers on the exam):

- Parsing assistant text to detect "Task completed" → WRONG
- Using `max_iterations=5` as the primary stopping condition → WRONG
- Checking if assistant produced text content as a signal → WRONG

1.2 Hub-and-Spoke Multi-Agent Architecture



Coordinator responsibilities:

- Decomposes task into subtasks — too narrow = missed coverage (exam trap!)
- Delegates to subagents via Task tool (must be in `allowedTools`)
- Aggregates & validates results; handles errors & retries
- ALL communication routes through coordinator (not direct agent-to-agent)

Critical: Subagents have ISOLATED context

- Subagents do NOT inherit coordinator's conversation history
- You MUST pass all needed context explicitly in the Task prompt
- Multiple Task calls in one response = parallel execution

```
# BAD - subagent has no context
Task: "Analyze the document"

# GOOD - full context in prompt
Task: "Analyze: [full doc text]. Prior results: [results]. Output: [schema]"
```

1.3 Hooks — Deterministic vs Probabilistic

Attribute	Hook vs Prompt
Guarantee	Hook = 100% deterministic Prompt = ~90%, not 100%
Use hooks for	Financial limits, compliance rules, policy enforcement
Use prompts for	General preferences, recommendations, formatting
Example	Block refund > \$500 → HOOK. "Try to resolve before escalating" → PROMPT

Hook Type	When it fires
PreToolUse	Before a tool call — can block/redirect the call
PostToolUse	After a tool call — normalize data before model sees it

1.4 Session Management

Mechanism	Purpose & When to Use
--resume <name>	Continue a named session. Beware: tool results may be stale if files changed
fork_session	Create independent branch from shared context. Use for comparing approaches
New session + summary	Better than resume when results are stale or time has passed

Domain 2: Tool Design & MCP Integration (18%)

2.1 Tool Descriptions — The #1 Selection Mechanism

The model selects tools PURELY based on descriptions. Bad descriptions = wrong tools chosen.

Description quality	Effect
Minimal: 'Retrieves customer info'	Model confuses overlapping tools → wrong selection
Good: includes what it returns, input formats, examples, edge cases, when NOT to use	Reliable selection even between similar tools

Include in every tool description:

- What the tool DOES and what it RETURNS
- Input formats and example values
- Edge cases and constraints
- When to use THIS tool vs similar alternatives

⚠ Exam trap: system prompt wording can create unintended tool associations (e.g., 'always verify customer' → overuse of get_customer)

2.2 tool_choice Parameter

Value	Behavior	When to Use
"auto"	Model decides: tool OR text	Default — most cases
"any"	Model MUST call some tool	Guarantee structured output
{"type": "tool", "name": "X"}	Model MUST call specific tool	Enforce execution order

2.3 JSON Schemas for Structured Output

tool_use + JSON Schema = most reliable structured output. Eliminates SYNTAX errors, NOT semantic errors.

Schema rule	Why
required only if data is always available	Required fields force hallucination when data is missing
type: ["string", "null"] for optional fields	Model returns null instead of fabricating
enum with "other" + detail field	Extensible — no data lost outside categories
enum "unclear" option	Honest uncertainty better than wrong category

Error type	Caused by	Fixed by
Syntax (invalid JSON, wrong type)	No schema	Use tool_use with JSON schema
Semantic (totals don't add up, wrong field)	Logic error	Validation + retry with feedback

2.4 MCP Configuration

Config location	Scope & purpose
.mcp.json (project root)	Team-shared, in VCS. Use for shared integrations
~/.claude.json (home dir)	Personal only, NOT in VCS. Use for experiments

```
// .mcp.json - store secrets as env vars, NEVER commit tokens
{ "mcpServers": {
  "github": {
    "command": "npx",
    "args": ["-y", "@modelcontextprotocol/server-github"],
    "env": { "GITHUB_TOKEN": "${GITHUB_TOKEN}" } // ← env var substitution
  }
}
```

2.5 MCP isError Flag

Generic errors give the agent nothing to work with. Always return structured error context:

```
// GOOD - structured error
{ "isError": true, "content": {
  "errorCategory": "transient",
  "isRetryable": true,
  "message": "Service temporarily unavailable",
  "attempted_query": "order_id=12345",
  "partial_results": null
}
}

// BAD - generic (gives agent nothing to decide with)
{ "isError": true, "content": "Operation failed" }
```

Error Category	Retryable?	Agent Action
Transient (timeout, 503)	Yes	Retry with exponential backoff
Validation (bad input format)	No (fix input)	Modify request and retry
Business (policy violation)	No	Explain to user; propose alternative
Permission (access denied)	No	Escalate

Domain 3: Claude Code Configuration & Workflows (20%)

3.1 CLAUDE.md Hierarchy

Level	Location	Shared via VCS?
User-level	~/.claude/CLAUDE.md	NO — personal only
Project-level	.claude/CLAUDE.md or root CLAUDE.md	YES — all contributors
Directory-level	CLAUDE.md in subdirectory	YES — that directory only

⚠ Classic exam trap: new team member doesn't receive instructions → reason is they were placed in user-level (~/.claude/CLAUDE.md) instead of project-level (.claude/CLAUDE.md)

- @path syntax: reference external files from CLAUDE.md (e.g., @./standards/testing.md)
- Max import nesting depth: 5

3.2 .claude/rules/ — Path-Conditional Rules

Rules load ONLY when editing files matching the glob pattern → saves context, reduces noise:

```
# .claude/rules/testing.md
---
paths: ["**/*.test.tsx", "**/*.test.ts"]
---
Tests must use describe/it blocks.
Do not mock the database — use a test database.
```

Use .claude/rules/ with paths when...	Use directory-level CLAUDE.md when...
Conventions apply to files ACROSS many directories (e.g., all tests)	Conventions tied to ONE specific directory
File-type-based patterns (*.test.tsx, terraform/**/*)	Subdirectory-specific context

3.3 Skills (.claude/skills/) and Slash Commands

Location	Scope
.claude/commands/ or .claude/skills/	Project — shared via VCS
~/.claude/commands/ or ~/.claude/skills/	Personal — NOT shared

SKILL.md frontmatter parameters:

Parameter	Value	Effect
context: fork	fork	Runs in isolated subagent — output doesn't pollute main session
allowed-tools	["Read","Grep","Glob"]	Restricts tools available to the skill

argument-hint	"Path to analyze"	Prompts for argument when invoked without one
---------------	-------------------	---

Use a Skill when...	Use CLAUDE.md when...
On-demand invocation for a specific task (review, analysis)	Always-loaded standards & conventions
Verbose output that would pollute main session	Architectural decisions for all sessions

Personal skill override: create ~/.claude/skills/commit/SKILL.md with the same name to override the team version for yourself only.

3.4 Planning Mode vs Direct Execution

Use Planning Mode when...	Use Direct Execution when...
Large changes across dozens of files	Single-file fix with clear stack trace
Multiple viable approaches (architecture decisions)	Adding one validation check
Unfamiliar codebase — must understand before changing	Simple, unambiguous change
Library migration touching 45+ files	Well-understood, small scope

Best practice: Planning mode for investigation → user approves plan → Direct execution to implement.

Explore subagent: runs discovery in isolated context → returns only a summary → protects main context window.

3.5 Claude Code CI/CD

```
# Non-interactive mode (the ONLY correct way for CI/CD)
claude -p "Analyze this pull request for security issues"

# Structured JSON output for inline PR comments
claude -p "Review this PR" --output-format json --json-schema '{...}'
```

- -p (--print): processes prompt, prints to stdout, exits. No interactive input.
- Session context isolation: generate code + review in SEPARATE instances — same session is biased.
- Re-review after commits: include prior review results in context, instruct to report only new/unresolved issues.
- Batch API NOT supported for iterative tool-calling workflows (multi-turn tool use requires synchronous).

Domain 4: Prompt Engineering & Structured Output (20%)

4.1 Few-Shot Prompting

Most effective way to produce consistently formatted output — more reliable than textual instructions.

Use few-shot examples for...	Effect
Ambiguous scenarios (tool selection, edge cases)	Shows decision logic unambiguously
Output formatting (location, issue, severity, fix)	Model generalizes pattern to new cases
Separating acceptable vs problematic code	Reduces false positives
Extraction from varied document formats	Handles non-standard structures

- Provide 2–4 targeted examples WITH rationale for why
- Target examples at the SPECIFIC ambiguous cases where errors occur
- Grouping by tool first (all get_customer examples, then all lookup_order) is LESS effective than showing the decision between them

4.2 Explicit Criteria vs Vague Instructions

Vague (bad)	Explicit (good)
"Check comments for accuracy"	"Flag a comment ONLY if it describes behavior that CONTRADICTS actual code behavior"
"Be conservative"	"Do NOT flag: stylistically outdated comments, minor wording inaccuracies, missing comments"
"Report high confidence only"	"CRITICAL: runtime failure. HIGH: security vuln. MEDIUM: logic bug no immediate impact. LOW: code quality"

4.3 Validation & Retry-with-Feedback

On validation failure, retry with **THREE** things:

- The original document
- The previous (incorrect) extraction
- The SPECIFIC error: e.g., 'Field total=150, but sum(line_items)=145'

Retry WILL help	Retry will NOT help
Format errors (date wrong format)	Information absent from the source document
Structural errors (field in wrong place)	Required context is in an external document
Arithmetic inconsistencies	Genuinely ambiguous / no ground truth

4.4 Message Batches API

Attribute	Value
Cost savings	50% vs synchronous
Processing window	Up to 24 hours (NO latency SLA guarantee)
Multi-turn tool calling	NOT supported
Correlation	Use custom_id to link request → response

Use Batch API for...	Use Synchronous API for...
Overnight tech-debt reports	Blocking pre-merge checks (developer waiting)
Weekly security audits	Interactive code review
Processing 10,000+ documents	Any workflow needing iterative tool calls
Nightly test generation	Real-time responses required

4.5 Multi-Pass Review Architecture

For PRs with 10+ files — single-pass causes attention dilution, inconsistent feedback, missed bugs:

Pass	Scope
Pass 1 (per-file)	Analyze each file individually for local issues
Pass 2 (integration)	Cross-file data flows, dependencies, inconsistencies

Independent review instance: Use a SECOND Claude instance without generation context for reviews. Same session = confirmation bias (model is less likely to challenge its own decisions).

Domain 5: Context Management & Reliability (15%)

5.1 Context Pitfalls

Problem	Cause	Fix
Lost-in-the-middle	Model misses content in middle of long inputs	Place KEY findings at START and END
Tool result accumulation	40+ fields returned when only 5 needed	PostToolUse hook to trim to relevant fields
Progressive summarization loss	Numbers, dates, percentages get vague	Extract critical facts to a persistent 'case facts' block
Context degradation in long sessions	Model refers to 'typical patterns' not specific classes	Use scratchpad files; spawn subagents for discovery

Case facts block pattern:

```

=== CASE FACTS (updated whenever a new fact appears) ===
Customer ID: CUST-12345
Order ID: ORD-67890
Issue: Damaged item on delivery
Customer Request: Full refund
Status: Pending manager approval
===
# Include this in EVERY prompt, outside the summarized history

```

5.2 Escalation Patterns

Escalate IMMEDIATELY when...	Do NOT escalate on...
Customer explicitly asks for a manager	First expression of dissatisfaction
Policy doesn't cover the request (gap/exception)	Sentiment analysis results
Agent cannot make progress after attempts	Model self-rated confidence (1–10) — unreliable
Multiple customer matches found (ask for more ID)	Automated classifier output alone

Pattern: acknowledge emotion → propose solution → escalate ONLY if customer reiterates request for human

5.3 Error Propagation in Multi-Agent Systems

Anti-pattern	Why it fails
Generic status 'search unavailable'	Coordinator can't decide how to recover
Empty result = success (silent suppression)	Coordinator thinks no matches exist
Abort whole workflow on one failure	Lose all partial results
Infinite retries inside subagent	Latency + wasted resources

Return structured error context:

- `failure_type`: transient | validation | permission | business
- `attempted_query`: what was tried
- `partial_results`: whatever was retrieved before failure
- `alternative_approaches`: suggestions for coordinator recovery
- `coverage_impact`: what topic/section is now missing

Key distinction:

Access failure (timeout)	Valid empty result (0 results)
Coordinator should decide: retry? Use partial? Skip?	This is a successful query — no matches exist
<code>isRetryable</code> : true	<code>isRetryable</code> : false (it worked, just no data)

5.4 Provenance & Attribution

- Always preserve claim → source URL → document name → publication date
- Conflicting stats: include BOTH values with attribution — never arbitrarily pick one
- Include dates: temporal differences ≠ contradictions (2023 data vs 2024 data)
- Render by content type: financial data → tables | news → prose | findings → lists
- Stratified sampling: even 97% overall accuracy can hide 40% errors in specific document types

⚡ Quick Decision Reference

Hooks vs Prompts

Situation	Use
Block refunds > \$500 (financial)	Hook (PreToolUse) — deterministic guarantee
'Try to resolve before escalating'	Prompt — probabilistic is fine
Normalize Unix timestamps from third-party MCP tool	Hook (PostToolUse) — can't modify tool
'Always call get_customer first'	Programmatic precondition — not just a prompt

Sync API vs Batch API

Use Synchronous when...	Use Batch API when...
Developer is WAITING (pre-merge check)	Result needed by TOMORROW (overnight report)
Interactive / real-time response required	Non-blocking, scheduled work
Iterative tool calling needed (multi-turn)	Bulk processing 1,000+ documents
Under 24-hour deadline is hard requirement	50% cost savings matters more than speed

Planning Mode vs Direct Execution

Use Planning Mode	Use Direct Execution
Monolith → microservices (dozens of files)	Single-file fix with clear stack trace
Unfamiliar codebase (understand first)	Adding one validation check
Multiple viable architectures	Simple, unambiguous change
Library migration touching 45+ files	CLAUDE.md says clear approach exists

Tool Description Quality — Checklist

- What does it do? What does it return?
- What are the input formats? Give an example value
- What are the edge cases and constraints?
- When should this be used vs similar alternatives?
- If two tools overlap semantically → RENAME to eliminate overlap

Context Hierarchy — What Loads When

Config	Loads...	VCS?
--------	----------	------

~/.claude/CLAUDE.md	Always (user-level only)	No
.claude/CLAUDE.md / root CLAUDE.md	Always (project-wide)	Yes
Subdirectory CLAUDE.md	When in that dir	Yes
.claude/rules/ with paths	Only when file matches glob	Yes
.claude/skills/	On demand (/command)	Yes
~/.claude/skills/	On demand, overrides project skill of same name	No

Common Exam Traps — Memorize These

When you see this in an answer choice → ELIMINATE IT

Wrong answer pattern	Why it's wrong
'Add a routing classifier before the agent'	Overengineering — fix tool descriptions first
'Use sentiment analysis for escalation'	Mood ≠ case complexity
'Self-rated confidence 1–10 for escalation'	Model can be confidently wrong
'Max iterations=5 as stopping condition'	Only stop_reason = end_turn is reliable
'Merge similar tools into one'	Reduces selection clarity; usually wrong
'Move both sync and batch to Batch API'	Batch not suitable for blocking checks
'Switch to larger model for attention issues'	Larger context ≠ better attention quality
'3 independent passes, flag if 2 agree'	Suppresses real bugs; doesn't fix root cause
'Add few-shot examples' as first step for wrong tool selection	Fix descriptions FIRST — that's the root cause
'Put guidelines in first user message'	Reduces their authority; system prompt is correct
'Use progressive summarization for exact values'	Numbers/dates get lost in summarization
'Agent-to-agent communication bypassing coordinator'	Loses observability and error control

The 'first step' principle:

Wrong tool selection → FIRST check/fix tool descriptions (not add classifiers, not add few-shot)

Instruction drift → FIRST inject periodic behavioral reminders (not restart session, not post-validate)

Context loss of numbers → FIRST extract to case facts block (not better summarization)

Team member missing CLAUDE.md → FIRST check level (user vs project)

Scenario-Specific Key Points

Scenario	Key Concept	Watch For
Customer Support	Programmatic precondition for tool order	Prompt alone doesn't guarantee order
Customer Support	Escalate on policy gap, not sentiment	Sentiment analysis is always wrong
Multi-agent Research	Coordinator decomposition too narrow	Subagents work fine but coverage is wrong
Multi-agent Research	Coordinator as central hub	Direct agent-to-agent bypasses oversight
Claude Code CI	-p flag for non-interactive	CLAUDE_HEADLESS / --batch don't exist
Claude Code CI	Independent instance for review	Same session = biased review
Code Generation	Project .claude/skills/ for team commands	~/.claude = personal only

Code Generation	context: fork prevents session pollution	Without it, verbose output fills context
Conversational AI	Include full message history every call	API is stateless — no server memory
Conversational AI	Periodic reminders combat instruction drift	Not new session, not post-validate

□ API & SDK Quick Reference

Claude API Request Structure

```
{
  "model": "claude-sonnet-4-6",          // or claude-opus-4-6, claude-haiku-4-5
  "max_tokens": 1024,
  "system": "You are a helpful assistant.", // system prompt (not in messages)
  "messages": [
    {"role": "user", "content": "Hi!"},
    {"role": "assistant", "content": "Hello!"}, // include full history
    {"role": "user", "content": "How are you?"}
  ],
  "tools": [...],
  "tool_choice": {"type": "auto"}        // auto | any | {type:tool,name:X}
}
```

AgentDefinition (Agent SDK)

```
agent = AgentDefinition(
  name="customer_support",
  description="Handles customer requests",
  system_prompt="You are a customer support agent...",
  allowed_tools=["get_customer", "lookup_order", "process_refund",
                 "escalate_to_human"], # principle of least privilege
  # For coordinator: add "Task" to allowed_tools
)
```

PreToolUse Hook — Block Policy Violations

```
@hook("PreToolUse")
def enforce_refund_limit(tool_call):
    if tool_call.name == "process_refund" and tool_call.args.amount > 500:
        return redirect_to_escalation(tool_call) # deterministic block
```

PostToolUse Hook — Normalize Data

```
@hook("PostToolUse")
def normalize_dates(tool_result):
    # Convert Unix timestamp → ISO 8601
    # Convert "Mar 5, 2025" → "2025-03-05"
    return normalized_result
```

JSON Schema Best Practices

```
{
  "type": "object",
  "properties": {
    "category": {
      "type": "string",
```



```
    "enum": ["bug", "feature", "docs", "unclear", "other"] // ← include
    "unclear"
  },
  "category_detail": {
    "type": ["string", "null"], // ← nullable for optional info
    "description": "Details if category = other or unclear"
  },
  "confidence": { "type": "number", "minimum": 0, "maximum": 1 }
},
"required": ["category"] // ← only truly required fields
}
```

Out-of-Scope Topics (will NOT be on exam)

- Fine-tuning or training custom models
- API authentication, billing, or account management
- Deploying/hosting MCP servers (infrastructure, networking, containers)
- Computer use (browser automation, desktop interaction)
- Image analysis / Vision capabilities
- Streaming API or server-sent events
- OAuth, API key rotation, authentication protocols
- Cloud-provider configurations (AWS, GCP, Azure)
- Prompt caching implementation details (beyond knowing it exists)
- Token counting algorithms or tokenization specifics

Good luck, Adil! You've got this. ☐